

Design Documentation

Interactive House - AI Smart Control System

Project's Github repo link:

<https://github.com/DanielJ0131/interactive-house>

Revision History

Date	Version	Description	Author
2026-02-24	1.0	Initial AI design including safety and sensors (D1, D2)	Anitta Antony
2026-02-24	1.0	Initial AI design including safety and sensors (D3, D4)	Md Abu Taher
2026-02-28	1.0	Smart Home code	Daniel Marcarini
2026-02-28	1.0	Speech Recognition Initial Design D5	Daniel Holderik
2026-03-15	1,0	Floating Speech control and revission	Daniel Holderik
2026 -03-16	1.1	Modify D1-D4 for Web	Anitta Antony Md Abu Taher
2026-03-16	1.1	Testing	Daniel Marcarini
2026-04-24	1.2	Initial Speech Recognition for web (D5)	Daniel Holderik
2026-05-08	1.3	Added final design updates and emergency button feature	Anitta Antony Md Abu Taher
2026-05-08	1.3	Updated D6 with implemented Jest automated testing and Istanbul coverage for web authentication and Firebase configuration	Daniel Marcarini
2026-05-21	1.4	Fully redid and finalized D5 and D6	Daniel Holderik
2026-05-23	Final	Updated final testing status and project completion evidence	Daniel Marcarini

Design item List

Requirement Name	Priority	Progress
D1. AI Architecture Overview & Component Design	Essential	Done
D2. Intent Output Format + Prompting Rules(Firebase AI)	Essential	Done
D3. AI chat UI Design (Mobile+Web) + API Contract	Essential	Done
D4. Error Handling, Reliability & State/Context Handling	Desirable	Done
D5. Speech Recognition Integration and Voice Navigation Design	Essential	Done
D6. Web application testing strategy for device-control integration	Desirable	Done
D7. Emergency Button and Emergency Assistance Flow Design	Essential	Done

Design Item Descriptions

D1 - AI Architecture Overview & Component Design

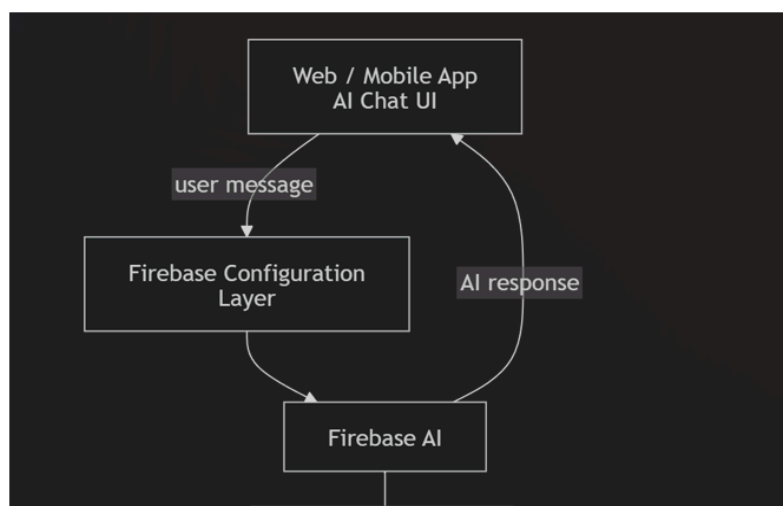
The AI feature is implemented for both web ([Next.js](#)) and mobile (React Native) using a shared Firebase configuration.

User messages are sent from the chat interface to Firebase AI, which generates responses and returns them to the UI.

The web client sends messages using direct AI requests, while the mobile client maintains a chat session for continuous conversation.

The UI displays the response and keeps chat history visible.

Figure 1:AI Architecture using Firebase AI for Web and Mobile Applications



D2 - Intent Output Format + Prompting Rules (Firebase AI)

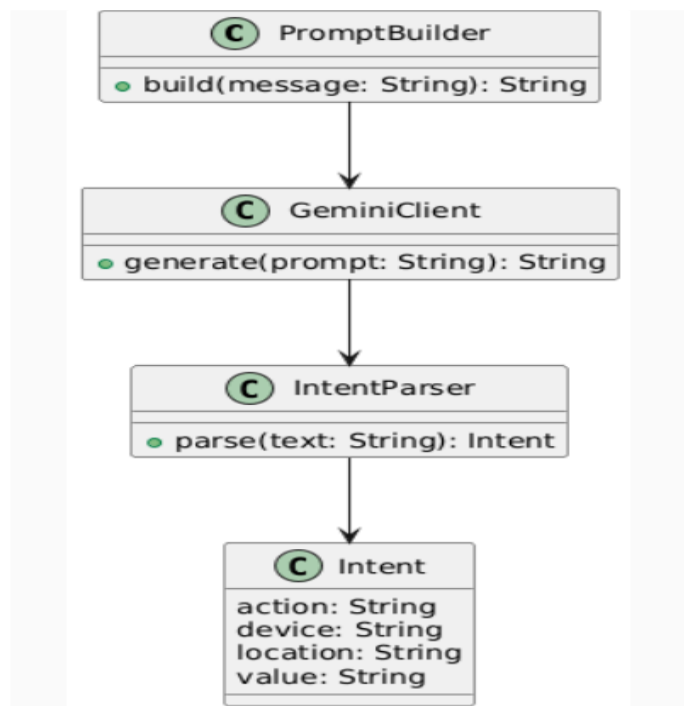
The AI system generates human-readable responses for the chat interface. Responses are used mainly for conversation, but the design also supports structured output for future smart-home commands.

Example intent JSON

```
{
  "action": "open",
  "device": "door",
  "location": "main",
}
```

Using Firebase AI allows flexible responses while keeping the system ready for command interpretation.

Figure – Design Class Diagram (AI intent pipeline)

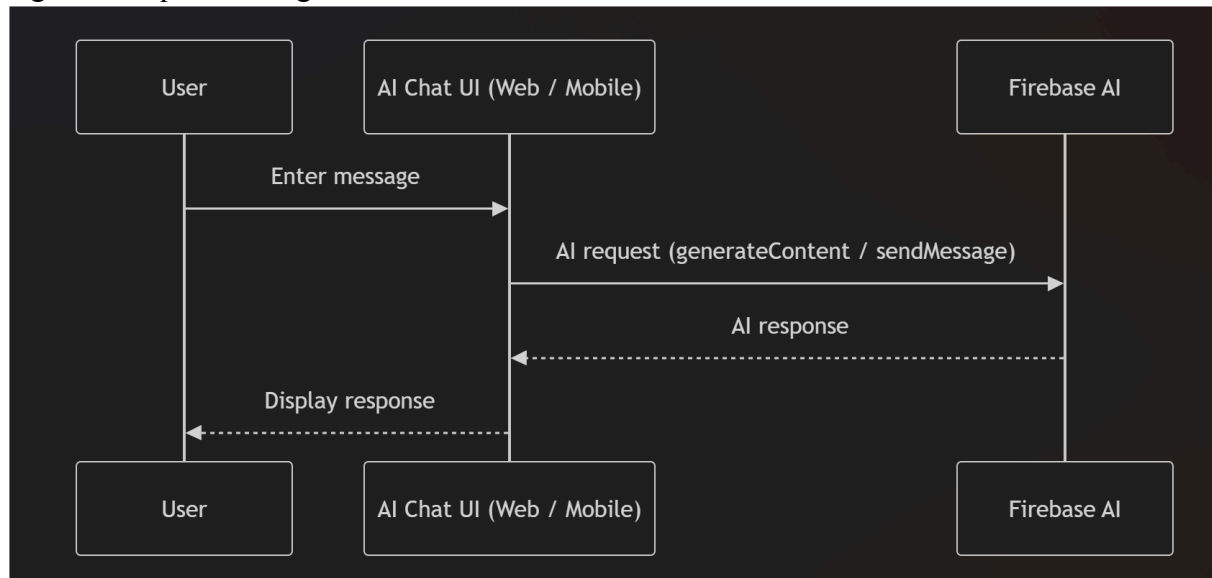


D3 - Mobile AI Chat UI Design (AiScreen.tsx) + API Contract

The AI chat interface is implemented in both the web and mobile applications. The UI keeps message history, allows user input, shows loading state, and displays AI responses. Web version uses `generateContent()` for each message. Mobile version uses a chat session to keep conversation context. Both versions use the same Firebase configuration.

This provides consistent behaviour across platforms

Figure – Sequence Diagram



D4 - Error Handling and Frontend State Management

Description

This design item explains how the frontend ensures reliability and safe interaction with the AI backend.

Reliability Rules

- Prevent empty message submission
- Prevent duplicate requests while loading
- Display configuration error if Firebase configuration is missing
- Show fallback message if AI request fails

Error Handling

- **Missing backend configuration:** Display setup warning
- **Network Error:** Show network error message
- **Invalid JSON response:** Handle safely without crashing
- **Non-200 response:** Display returned error message

State Management

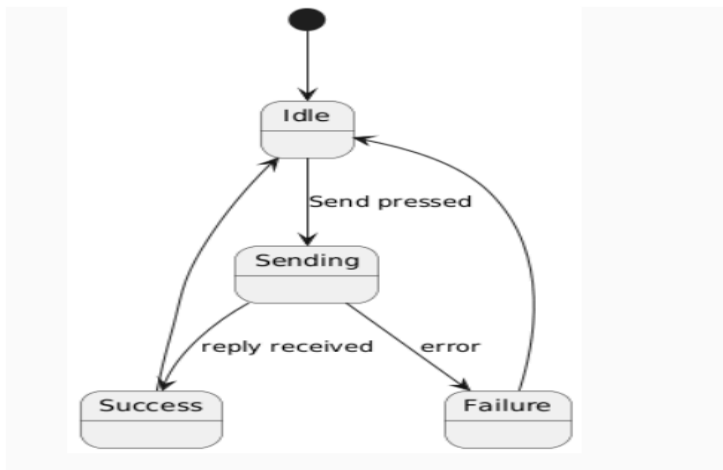
The frontend manages request states using a loading flag:

- Idle
- Sending
- Success
- Failure

This allows the AI system to correctly interpret context-dependent commands such as “Turn it off” by identifying which device is being referenced.

Maintaining device states ensures consistent behavior and improves the intelligence and usability of the AI system.

Figure – State Diagram



D5 Speech – Recognition Integration and Voice Navigation Design

This design item describes how speech recognition functionality is integrated into the mobile version of the Interactive House application. The design focuses on voice-based interaction, navigation control, and speech-driven device interaction within the React Native mobile environment.

The implementation allows users to interact with the system using spoken commands instead of relying only on manual touch-based interaction.

Technology and Build Configuration

The mobile speech-recognition system uses an external speech-recognition library integrated through Expo development builds. Native microphone access and real-time speech processing are required, which are not fully supported within the standard Expo Go environment.

A custom Expo development build was therefore configured to support:

- Native microphone access
- Real-time speech processing
- Speech recognition event handling
- Integration with the application navigation system

Speech Recognition Architecture

The speech-recognition system is integrated into the mobile application as a reusable interaction module connected to the existing navigation and control architecture.

The implementation captures spoken input, converts speech into text, and allows interpreted commands to trigger navigation or smart-device actions.

Global Voice-Control Integration

The original dedicated speech-recognition screen was redesigned and replaced with a persistent floating speech-control interface. The floating microphone component remains accessible while navigating between screens and allows speech interaction without requiring users to open a separate feature page.

The floating speech-control component:

- Remains visible across supported application screens
- Captures spoken commands through the speech-recognition library
- Displays recognized transcript feedback to the user
- Supports voice-triggered navigation actions
- Supports speech-triggered device-control interaction
- Uses centralized speech state handling across screens

Interaction Flow

1. The user activates the floating microphone interface.
2. The speech-recognition module captures spoken input.
3. Spoken input is converted into text.
4. The recognized transcript is processed by the application.
5. The system interprets the command and determines the requested action.
6. Navigation actions or device-control functions are triggered.
7. UI components update based on the resulting application state.

Voice-Control Integration

Speech commands are connected to the same underlying control logic used by the manual mobile interface. This ensures that speech interaction remains synchronized with the existing smart-device workflow and Firebase-backed state management.

Current implemented voice interactions include:

- Navigation between application screens
- Device toggle interaction
- Smart-device control actions
- Floating global voice activation
- Transcript feedback during speech recognition

Advantages

- Improved accessibility and hands-free interaction
- Persistent voice interaction across screens
- Integration with existing mobile navigation flow
- Reuse of existing UI control logic

- Real-time speech feedback through transcript display
- Modular speech architecture for future AI expansion

D6 – Web application testing strategy for device-control integration

This design item describes how the web application and voice-control system are verified during the current sprint. The testing focus is the interaction path between the web user interface, Firebase Authentication, Firestore synchronization, Web Speech API integration, and the smart-device control workflow used by the project.

Web Speech API Integration and Global State Management

The speech recognition system was redesigned to use the browser-based Web Speech API. This allows speech processing to occur directly within supported web browsers without requiring native mobile modules or a custom development build.

Technology Change

The Web Speech API provides real-time speech-to-text functionality in supported browsers such as Google Chrome and Microsoft Edge. This removes the dependency on external native speech-recognition libraries for the web platform and simplifies deployment and testing within browser environments

Global State Integration

Speech recognition is integrated into the application through centralized global state management. The recognized speech transcript is shared across screens and components, allowing voice commands to trigger actions regardless of the currently active page. The global state system acts as a shared communication layer between the floating speech-control component and the rest of the application interface.

Floating Voice-Control System

The application uses a persistent floating microphone interface that remains accessible across all supported pages. The floating component allows users to activate speech recognition without navigating to a dedicated speech screen.

The floating speech component:

- Captures spoken input through the Web Speech API
- Displays live transcript feedback to the user
- Supports navigation commands between application pages
- Supports voice-triggered interaction with existing UI controls
- Automatically stops listening after command execution to reduce repeated actions

Interaction Flow

1. The user activates the floating microphone icon.
2. The Web Speech API captures and processes spoken input.
3. The recognized speech transcript is stored in global state.
4. The application interprets the command.
5. The corresponding navigation action or device-control function is executed.
6. UI components update automatically based on the resulting state changes.

Advantages

- No dependency on native speech libraries for web deployment
- Simplified browser-based deployment and testing
- Centralized voice processing through shared global state
- Consistent interaction across multiple screens
- Reuse of existing UI control logic for speech commands
- Improved accessibility and hands-free interaction support

Browser Compatibility

The current implementation depends on browser support for the Web Speech API. Full functionality is currently targeted toward Chromium-based browsers such as Google Chrome and Microsoft Edge. Unsupported browsers may not provide reliable speech-recognition behavior.

Implemented Automated Testing Setup

The web application now includes an automated testing setup using Jest and React Testing Library. The test environment is configured for the Next.js frontend and uses jsdom to simulate browser behavior during test execution.

The automated tests are designed to verify frontend behavior without using the real Firebase backend. Firebase Authentication and Firestore functions are mocked during testing, which makes the tests repeatable, safe, and independent from external database state.

The current automated implementation covers the authentication part of the web application and the Firebase configuration layer. The login page tests verify rendering, empty input validation, successful login behavior, redirection to the hub page, and friendly handling of Firebase login errors. The signup page tests verify rendering, form validation, successful account creation flow, profile update, Firestore user document creation, and handling of already registered email errors.

The Firebase configuration tests verify that the application exports the Auth and Firestore instances correctly. This gives basic confidence that the shared Firebase configuration used by the web application is available to the frontend modules.

Coverage reporting is generated through Istanbul using Jest coverage. The report is available as an HTML page and shows which files and branches are covered by the current automated tests.

Current automated testing status:

- 4 test suites passing
- 14 tests passing
- Login screen tested
- Signup screen tested
- Firebase Auth configuration tested
- Firebase Firestore configuration tested
- Istanbul coverage report generated

The next design/testing step is to extend automated coverage to the hub page, device toggles, Firestore snapshot synchronization, emergency page, music page, and AI page.

Final Implementation Status

The web application testing strategy was completed with an automated Jest and React Testing Library test suite. The suite verifies the most important web flows, including authentication, Firebase configuration, shared UI components, static pages, guest hub, emergency flow, hub device-control behavior, and music page behavior.

The tests use mocked Firebase Authentication, Firestore, realtime snapshots, Next.js navigation, and browser AudioContext. This keeps the tests repeatable and independent from production Firebase data.

Final result:

- 9 passing test suites
- 49 passing tests

- Istanbul coverage report generated

The implemented test suite provides evidence that the core web application behavior can be verified repeatedly before demonstration and final delivery.

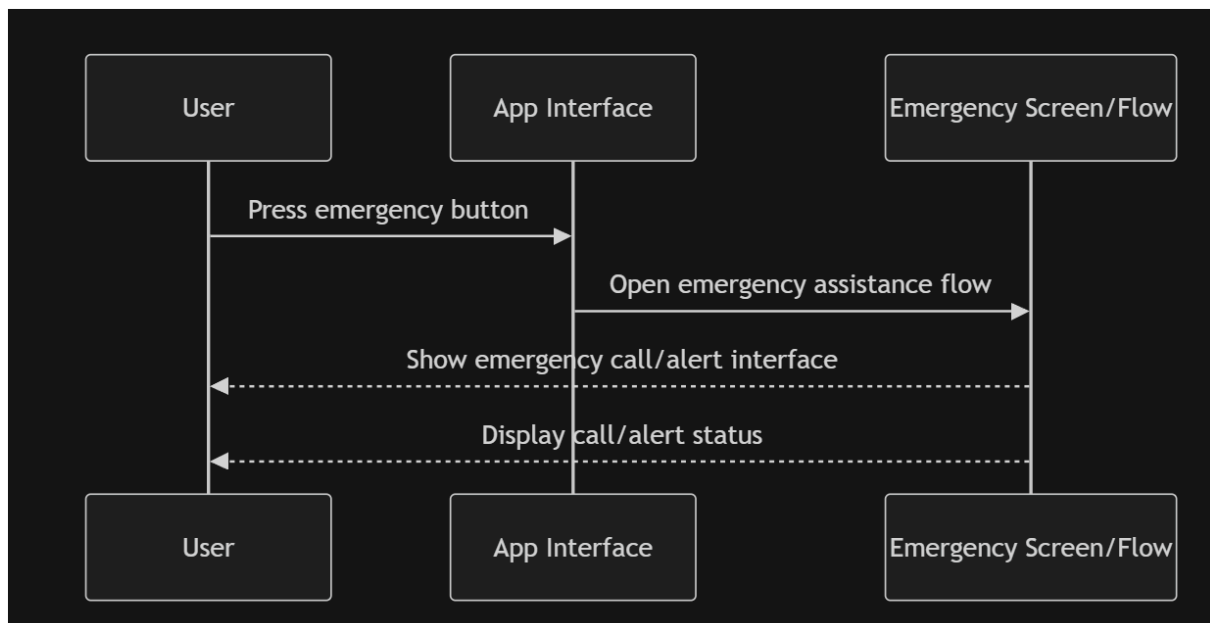
D7 – Emergency Button and Emergency Assistance Flow Design

Description

This design item describes the emergency assistance feature in the Interactive House application. The purpose of the feature is to provide users with quick access to an emergency action from the user interface. This is especially important for elderly or vulnerable users who may need immediate assistance during an unsafe situation.

Design Overview

The emergency button is designed as a visible and easily accessible interface element. When the user activates the emergency button, the application opens an emergency assistance flow. Since the project does not integrate with real emergency services, the feature is implemented as a simulated emergency call or alert flow for demonstration purposes.



This diagram shows that:

1. The user presses the emergency button
2. The application interface opens the emergency flow

3. The emergency screen appears
4. The user sees a simulated emergency call or alert status

Daniel Marcarini - Smart Home code

Requirement Name	Priority
D1. Embedded system architecture and hardware integration	Essential
D2. Sensor monitoring and state machine design	Essential
D3. Actuator control and alarm management system	Essential
D4. LCD display and user interaction handling	Essential
D5. Non-blocking timing and concurrent operation design	Essential

D1 – Embedded System Architecture & Hardware Integration

This design item defines the overall architecture of the embedded Smart Home system and the integration of hardware components.

The system is built around the Arduino Uno microcontroller, which serves as the central processing unit. All sensors, actuators, and interface components are connected directly to the microcontroller.

The architecture separates system functionality into three logical layers:

- Input Layer (gas sensor, temperature sensor, LDR, push buttons, Bluetooth)
- Processing Layer (state management and decision logic)
- Output Layer (servo motors, buzzer, LEDs, LCD display)

This layered design improves clarity and modularity while maintaining simplicity suitable for embedded systems.

D2 – Sensor Monitoring and State Machine Design

This design item specifies how environmental data is processed and how system states are managed.

The system continuously monitors:

- Gas levels
- Temperature
- Ambient light

Gas detection is implemented using a threshold-based comparison combined with a staged state machine. The state machine ensures that safety actions are executed in a controlled sequence rather than simultaneously.

Internal state variables maintain:

- Window open/closed state
- Door open/closed state
- Alarm activation status
- Gas detection transition state

This prevents repeated triggering and ensures stable system behavior.

D3 – Actuator Control and Alarm Management System

This design item describes how mechanical and alarm components are controlled.

The system controls:

- Window servo motor
- Door servo motor
- Buzzer (multiple modes)
- LED indicators

Buzzer behavior is centralized using defined operational modes (e.g., off, solid alarm, siren mode). This prevents conflicting sound activation from different subsystems.

During emergency events:

- The buzzer is activated
- The window may open automatically
- Visual indicators reflect hazard status

Safety-related actions override normal user interaction.

D4 – LCD Display and User Interaction Handling

This design item defines how the system provides feedback and processes user input.

The LCD module displays:

- Environmental readings
- Alarm notifications
- Device status information

Display updates are controlled to prevent flickering and unnecessary refresh operations.

User interaction is implemented using:

- Push buttons with edge-detection logic
- Bluetooth communication for remote commands

Button presses are detected through state comparison to ensure reliable toggling behavior.

D5 – Non-Blocking Timing and Concurrent Operation Design

This design item explains how the system maintains real-time responsiveness.

The system uses non-blocking timing mechanisms based on internal timers instead of delay-based blocking. This allows:

- Continuous sensor monitoring
- Immediate emergency response
- Simultaneous user interaction handling
- Stable LCD updates

This design ensures that no subsystem prevents another from executing, maintaining reliable concurrent behavior within the embedded environment.